DriveAlert: A Camera-Based Driver Drowsiness Detection and Multi-Modal Alert System

Abu Sayem
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
abu.sayem@stud.hshl.de

Tajbid Ahmed
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
tajbid.ahmed@stud.hshl.de

Md. Tawhidur Rahman Tuhin
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
md-tawhidur-rahman.tuhin@stud.hshl.de

***Abstract*—Driving while tired is one of the biggest causes of road accidents, and new cars in the European Union are now required to have a Driver Drowsiness and Attention Warning (DDAW) system. In this project we built a complete SysML model of DriveAlert, a system that uses an in-cabin infrared camera and an electronic control unit (ECU) to detect when a driver is getting sleepy and to warn them in time. The system reads fatigue cues such as PERCLOS, blinking, yawning and head-nodding, classifies the driver as Awake, Mildly Drowsy or Severely Drowsy, and gives a step-by-step warning that goes from a dashboard light, to a sound, to a vibration, and finally to an emergency call if the driver does not react. This paper presents the full model in eight SysML diagrams: the requirement diagram, the use case diagram, the block definition diagram (BDD), the internal block diagram (IBD), the activity diagram, the state machine diagram, the sequence diagram and the parametric diagram. We also show how the views fit together so that every requirement is covered by the architecture and the behaviour.**

I. Introduction

A modern car carries a lot of responsibility for keeping the driver safe, and one of the most dangerous situations is a driver who is falling asleep at the wheel. This happens most on long highway trips and during night driving, and many serious accidents are caused not by a mechanical fault but simply because the driver lost focus or fell asleep for a few seconds. Because of this, DDAW systems are now required for new cars in the EU, and companies such as Bosch, Continental and HELLA/Forvia already build them. We chose DriveAlert as our project because it is a real and meaningful problem, not just a theoretical one.

The idea behind DriveAlert is simple: the car watches the driver and warns them if they become too tired to drive safely. An infrared camera inside the cabin points at the driver's face and sends video to an ECU, which checks for signs of tiredness and decides how drowsy the driver is. Based on that, the system gives the right kind of warning, starting small and getting stronger if the driver does not react. An important point in our design is that DriveAlert only watches and warns; it never takes over the steering, the brakes or the engine, so the driver is always in full control.

To design the system properly we used Model-Based Systems Engineering (MBSE) with SysML. This paper presents the complete model in Section III, where each subsection covers one diagram: the requirement diagram, the use case diagram, the block definition diagram and the internal block diagram, followed by the activity diagram, the state machine diagram, the sequence diagram and the parametric diagram. Section IV describes the modelling tool, Section VI explains the traceability between the views, and Section VII concludes.

II. System Overview

DriveAlert works like a quiet observer that only speaks up when it is needed. The infrared camera runs the whole time the car is on and films the driver's face, even at night thanks to its near-IR lighting. The ECU is the brain of the system: it processes the camera images every 50 ms and looks for fatigue cues such as PERCLOS (the percentage of time the eyes are closed), the blink rate and duration, yawning, and the head slowly dropping forward. The steering angle and the lane position from the car are used as a second check, so that a single long blink does not trigger a false alarm on its own.

From these signals the ECU decides whether the driver is Awake, Mildly Drowsy or Severely Drowsy, and it answers each state with a different level of warning. The warnings escalate in stages: first a visual alert on the dashboard, then a sound from the buzzer, then a vibration in the steering wheel or the seat, and in the severe state the system also speaks to the driver and shows the nearest rest stop on the navigation display. If the driver still does not react, the system enters an emergency state, makes an automatic emergency call (eCall) with the GPS location, and flashes the hazard lights to warn the other cars. If a component such as the camera fails, the system goes into a degraded mode, turns off the alerts, shows a fault icon and stores a diagnostic trouble code (DTC) so a mechanic can find the problem later.

III. SysML Diagrams

We modelled DriveAlert with SysML so that the requirements, the behaviour and the structure all stay connected and traceable. This section presents the eight diagrams of the model.

A. Requirement Diagram

The requirement diagram in Figure 1 describes what the system must do, without saying yet how it is built. Each

requirement is shown as a box with the «requirement» stereotype and carries an identifier and a short text. We used the containment relationship (the cross symbol) to break a large requirement into smaller ones that can actually be tested. There is one main stakeholder requirement at the top, R0 (*Fatigue accident prevention*), and all the other requirements are contained under it.

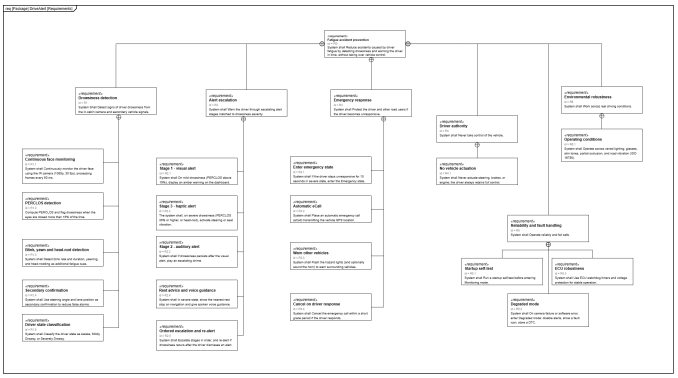

Fig. 1. SysML requirement diagram for DriveAlert.

The requirements are organised into six groups. **R1 (Drowsiness detection)** covers the continuous camera monitoring, the PERCLOS calculation, the detection of blinks, yawning and head-nodding, the secondary confirmation from the steering and lane signals, and the classification of the driver state. **R2 (Alert escalation)** covers the three warning stages (visual, auditory, haptic), the rest-stop and voice guidance in the severe state, and the rule that the stages always run in order and re-alert if the driver becomes drowsy again. **R3 (Emergency response)** covers entering the emergency state after the driver is unresponsive for 10 s, placing the automatic eCall, warning the surrounding vehicles, and cancelling the call within a grace period if the driver reacts. **R4 (Driver authority)** is the safety constraint that the system must never use the steering, brakes or engine. **R5 (Reliability and fault handling)** covers the startup self-test, the degraded mode and the ECU watchdog and voltage protection, and **R6 (Environmental robustness)** requires the system to work with different lighting, glasses, skin tones, partial occlusion and road vibration. We kept the same threshold values everywhere (PERCLOS above 15% for mild, 30% or more for severe, and the 10 s timeout) so that the requirements match the other diagrams.

B. Use Case Diagram

The use case diagram in Figure 2 shows the system from the point of view of the people and things outside it. The system itself is drawn as a boundary box, the use cases are the ellipses inside it, and the actors are the stick figures outside. The Driver is the primary actor, because everything starts from the person driving the car. There are three secondary actors: the Emergency Call Center, which receives the automatic eCall and the GPS location; the Surrounding Vehicles, which receive the hazard-light warning; and the Service Technician, who reads the stored DTC after a fault. We did not model the camera, the ECU or the sensors as actors, because they are inside the system and belong in the structural diagrams instead.

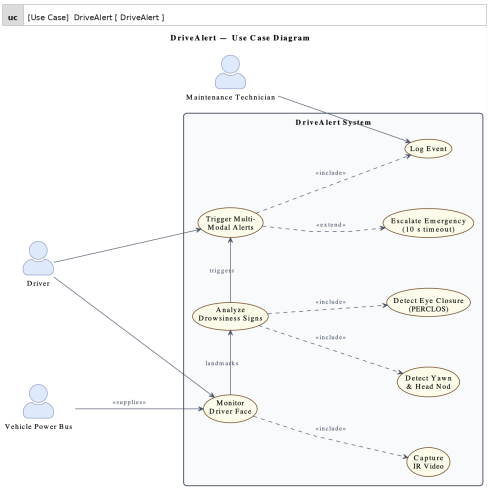

Fig. 2. SysML use case diagram for DriveAlert.

The use cases are connected with two kinds of dependencies. We used «include» for behaviour that always happens as part of another use case, and «extend» for behaviour that only happens sometimes. *Monitor driver drowsiness* includes *Detect fatigue signs*, because the detection always runs as part of the monitoring. *Guide to nearest rest stop* extends *Alert driver*, because the rest-stop guidance only happens in the severe state. *Handle unresponsive driver* also extends *Alert driver*, since it only triggers when the driver does not react, and it in turn includes *Place emergency eCall* and *Warn surrounding vehicles*, because once the emergency handling starts both of these always happen. This structure matches exactly how the system escalates from a simple warning to a full emergency response.

C. Block Definition Diagram

The block definition diagram (BDD) in Figure 3 shows what the system is made of. Each component is a block with the «block» stereotype. We used two kinds of relationships. Composition, shown by the filled black diamond on the system side, is used for the twelve components that DriveAlert really owns: the IR driver camera, the steering/lane input, the drive-time monitor, the driver monitoring ECU, the dashboard display, the buzzer/chime, the haptic vibration motor, the navigation link, the speaker with voice output, the telematics unit with GPS, the exterior lights and horn link, and the power supply. Each of these parts belongs to only one system and exists only as long as the system does, so the multiplicity is 1.

For the CAN bus and Ethernet we used a **reference association** (a plain line) instead of composition, because the communication network is part of the car and DriveAlert only uses it, it does not own it. Each composition end also carries

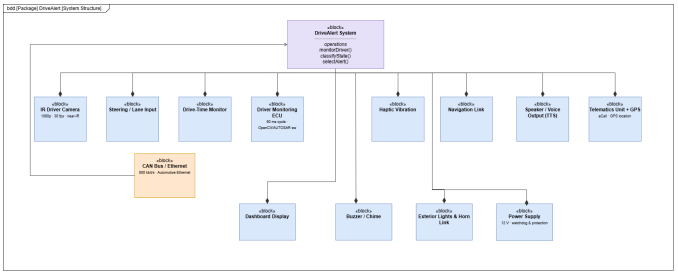

Fig. 3. SysML block definition diagram for DriveAlert.

a role name, such as `cam` for the camera, `ecu` for the ECU and `telematics` for the telematics unit. These role names are not just decoration; we use the same names again in the internal block diagram, so that the structure and the wiring stay consistent across the two diagrams.

D. Internal Block Diagram

While the BDD shows what parts exist, the internal block diagram (IBD) in Figure 4 shows how they are connected to each other. The driver monitoring ECU sits in the middle and works as the hub, so every part connects to it rather than to the other parts directly. The connections are made through ports, which are the small squares on the edges of the blocks, and the connectors between the ports have a type that says which interface is used.

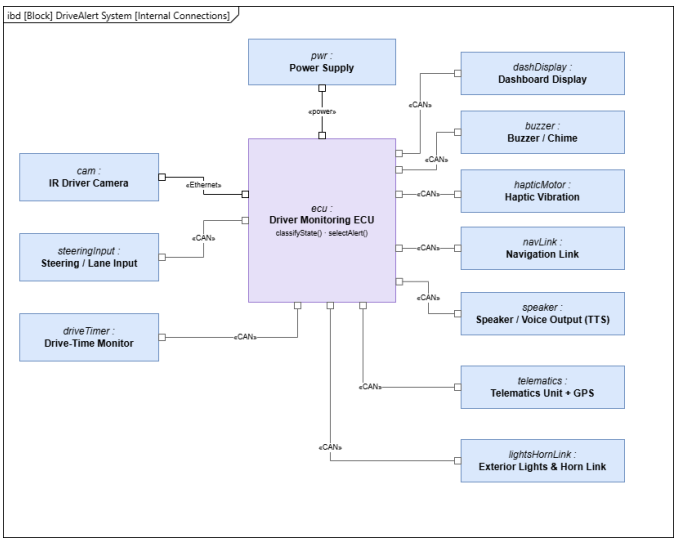

Fig. 4. SysML internal block diagram for DriveAlert, with the ECU as the hub.

Following our system concept, the camera link uses Automotive Ethernet, because the live video is too much data for the CAN bus, while all the control signals use the CAN bus at 500 kbit/s. Each connector also carries an item flow, drawn as a solid arrow that names what is sent and points to where it goes. The camera sends `videoFrames` into the ECU over Ethernet, while the steering/lane input and the drive-time monitor send their data into the ECU over CAN, and the power supply provides the 12 V. In the other direction, the ECU sends control commands out to the alert devices: the visual command to the dashboard, the chime command to the buzzer, the haptic command to the vibration motor, the rest-stop data to the navigation link, the voice message to the speaker, the eCall trigger to the telematics unit, and the hazard command to the lights and horn link.

E. Activity Diagram

The activity diagram in Figure 5 shows the operational flow of the system, that is, the order in which the actions happen while DriveAlert is running. The flow begins when the ignition is on: the IR camera captures a video frame every 50 ms and passes it to the ECU, which extracts facial landmarks. Two analysis branches then run in parallel — one computes PERCLOS from the eye-closure ratio, and the other detects yawn events and head-nod cycles. The parallel results are fused by the ECU into a single weighted drowsiness score.

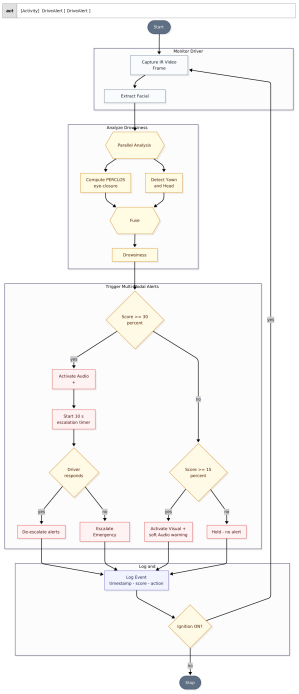

Fig. 5. SysML activity diagram for DriveAlert.

The score is then passed to the alert-selection logic. If the score is below 15% the system holds with no alert. If it reaches the mild threshold (15%) the ECU activates a visual cue and a soft audio warning. If it reaches the severe threshold (30%) the ECU activates all three channels — visual, audio and haptic — and starts a 10 s escalation timer. Should the driver respond in time, the alerts de-escalate and silent monitoring resumes. If no response is received, the system escalates to maximum-intensity continuous alerts. Every iteration ends by logging a timestamped event record before the loop checks whether the ignition is still on.

F. State Machine Diagram

The state machine diagram in Figure 6 shows the discrete states the system moves through and the guard conditions and events that trigger each transition. On power-on the system enters the *Initializing* composite state, which sequences through *Booting*, *SelfTest* and *Calibrating* sub-states before reaching *Ready*. Once initialisation is complete the system enters the *Monitoring* composite state, which contains the four operational sub-states: *Normal*, *MildAlert*, *SevereAlert* and *Emergency*.

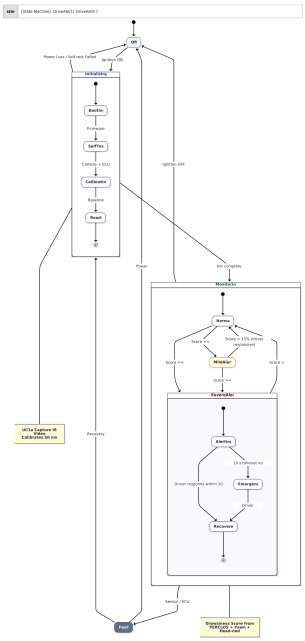

Fig. 6. SysML state machine diagram for DriveAlert.

Inside Monitoring, the system starts in *Normal* and moves to *MildAlert* when the drowsiness score reaches 15%. From *MildAlert* it moves to *SevereAlert* when the score reaches 30%, and the driver can also jump directly from *Normal* to *SevereAlert* if the score crosses 30% without passing through the mild stage. If the driver responds within 10 s the system de-escalates back to *Normal*. If no response is received the system transitions to the *Emergency* sub-state, which triggers maximum-intensity alerts. At any point, a sensor or ECU failure sends the system to the *Fault* state outside the Monitoring region; from *Fault* the system can return to *Initializing* via a recovery attempt, or go back to *Off* via a power cycle. Ignition-off from *Normal* also returns the system to *Off*.

G. Sequence Diagram

The sequence diagram in Figure 7 shows the time-ordered messages exchanged between the system components for the complete escalation scenario. The lifelines are the Driver, the IR Camera, the ECU Processor, the Alert Module (audio, visual, haptic) and the Event Log, with the Maintenance Technician appearing at the end for offline diagnostics. The diagram is structured as one outer loop running at the 50 ms camera cycle, with two nested alternative fragments: one for the mild-alert case and one for the severe-alert and emergency escalation.

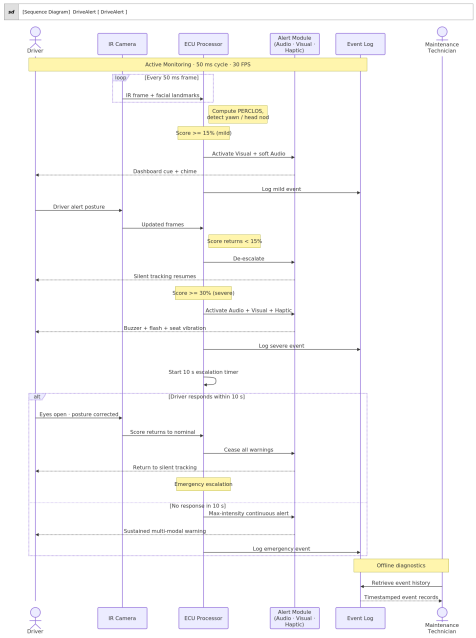

Fig. 7. SysML sequence diagram for DriveAlert.

In each loop iteration the camera sends an IR frame with facial landmarks to the ECU. When the ECU computes a score at or above 15% it sends an activate command to the Alert Module, which returns a dashboard cue and a chime to the Driver; the ECU simultaneously logs a mild event to the Event Log. If the driver corrects their posture the score drops below 15%, the ECU sends a de-escalate command and silent tracking resumes. If the score reaches 30% the ECU activates all three alert channels, logs a severe event, and starts an internal 10 s self-message timer. The *alt* fragment then splits into two branches: in the first the driver responds within 10 s, the score returns to nominal, the ECU ceases all warnings and returns to silent tracking; in the second no response arrives, the ECU sends a max-intensity continuous-alert command, logs an emergency event, and the system remains in the escalated state. After the session, the Maintenance Technician sends a retrieve-history request to the Event Log, which replies with timestamped event records.

H. Parametric Diagram

The parametric diagram in Figure 8 shows the mathematical constraints and value bindings that govern the drowsiness-score computation and the alert-level decision. The diagram is organised into six constraint blocks arranged in a left-to-right dataflow. On the far left, the *Physical Parameters* block supplies the raw measurements: total frame count at 30 FPS, closed-eye frame count, yawn count, head-nod count

and driver response time. These feed into the *Drowsiness Constraint* block, which computes PERCLOS as the ratio of closed-eye frames to total frames multiplied by 100, and derives the yawn rate and nod rate by dividing event counts by the observation window.

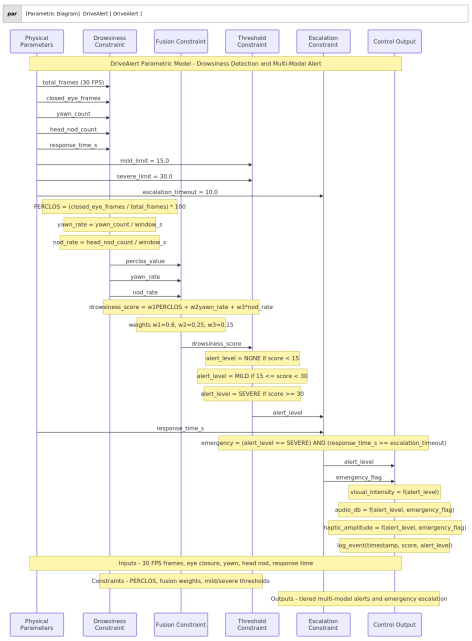

Fig. 8. SysML parametric diagram for DriveAlert.

The three computed signals pass to the *Fusion Constraint* block, which applies the weighted sum:

$$\text{score} = w_1 \cdot \text{PERCLOS} + w_2 \cdot \text{yawn\_rate} + w_3 \cdot \text{nod\_rate}$$

with weights $w_1 = 0.6$, $w_2 = 0.25$ and $w_3 = 0.15$. The resulting score flows into the *Threshold Constraint* block, which maps it to an alert level: NONE if the score is below 15, MILD if it is between 15 and 30, and SEVERE if it is 30 or above, using the limit values supplied by the *Drowsiness Constraint* block (mild_limit $= 15$, severe_limit $= 30$). The alert level and the driver response time are then passed to the *Escalation Constraint* block, which raises an emergency flag when the alert level is SEVERE and the response time meets or exceeds the 10 s escalation timeout. Finally, the *Control Output* block computes the visual intensity, audio level in dB and haptic amplitude as functions of the alert level and the emergency flag, and the Event Log records a timestamped entry for every state change.

IV. Modelling Tool

We created the diagrams in draw.io (diagrams.net), a free online diagramming tool. We used its UML/SysML shape library for the actors, blocks, ports and connectors, and for the requirement and structural diagrams we also used the CSV import feature, which lets us define the boxes and their relationships as a text table and generate the diagram automatically. This was useful for keeping the requirement identifiers, the role names and the connections consistent, and for redrawing a diagram quickly when we changed something. The diagrams were then exported as images and included in this report.

V. Team Collaboration

A detailed breakdown of individual contributions and task distribution is documented in the project repository. For the complete collaboration log, please refer to the following link: https://github.com/CCsayem/DriveAlert_ISE2_GroupE/blob/main/Team_Contribution/Teamwork.PNG.

VI. Traceability

One of the main reasons for using SysML is that the different views stay connected, so we made sure our diagrams trace to each other. The requirements are the starting point: the detection requirements in R1 are satisfied by the camera, the ECU and the input components and are exercised by the *Monitor driver drowsiness* and *Detect fatigue signs* use cases. The alert requirements in R2 are satisfied by the ECU together with the dashboard, buzzer, haptic motor, navigation link and speaker, and are exercised by the *Alert driver* and *Guide to nearest rest stop* use cases. The emergency requirements in R3 are satisfied by the ECU, the telematics unit and the lights/horn link and are exercised by the *Handle unresponsive driver*, *Place emergency eCall* and *Warn surrounding vehicles* use cases, and the same R3.1 condition (unresponsive for 10 s in the severe state) is the transition into the Emergency state in the state machine diagram. The safety constraint R4 applies to the whole system, R5 is satisfied by the ECU and the power supply and is exercised by the *Report system fault* use case and the Fault state, and R6 is satisfied by the camera. Going through the requirements this way shows that every requirement is covered by at least one block, one use case and the matching behaviour, and that nothing is left out.

VII. Conclusion

In this project we built a complete SysML model of DriveAlert. The requirement diagram describes what the system has to do and breaks it down into testable requirements, the use case diagram shows what the system does for the driver and the other actors and how it escalates from a warning to an emergency, and the BDD and IBD show what the system is made of and how the parts are connected around the central ECU. The activity, state machine and sequence diagrams describe the behaviour of the system over time, and the parametric diagram defines the values and constraints behind the decisions. The role names from the BDD are reused in the IBD so the two diagrams match, and the traceability check confirms that every requirement is covered by the structure and the behaviour. Together these eight diagrams form a full model of the DriveAlert system and a good basis for a real implementation.